SSN College of Engineering – Kalavakkam

Department of Computer Science and Engineering

ICS1513 – Computer Networks Laboratory

Experiment 6: Implementation of Error Detection Using Checksum and CRC
Name: Saravana Senthilkumar | **Roll No:** 3122247001057 | **Date:** 17.08.2026 / 19.08.2026

Objective

To develop socket programming applications in C that simulate reliable data transmission over network streams using:

1. **1's Complement Checksum Algorithm** (8-bit word division, end-around carry addition, and complement generation/verification).
2. **Cyclic Redundancy Check (CRC)** using generator polynomial $G(x) = x^4 + x + 1$ (binary 10011) with socket-level transmission and manual error injection.

Part A: Checksum Error Detection

Mathematical Principle

1. The sender splits data bytes into 8-bit words ($B_1, B_2, \ldots, B_n$).
2. All 8-bit words are added sequentially. Any carry bit beyond 8 bits ($> 0\text{xFF}$) is added back to the sum (end-around carry):

$$\text{Sum} = \left(\sum B_i\right)_{\text{end-around carry}}$$

3. The sender computes the 1's complement of the sum: $\text{Checksum} = \sim \text{Sum}$.
4. The frame transmitted consists of the payload data bytes appended with the Checksum byte.
5. At the receiver, all incoming bytes (data + checksum) are summed using 1's complement addition.
6. Verification rule: If the final verification sum is 0xFF , no bit errors occurred. Any other value indicates transmission corruption.

Algorithms

Checksum Sender Algorithm

1. Read input string message.
2. For each 8-bit byte, add to sum. If sum > 0xFF, perform end-around carry: sum = (sum & 0xFF) + 1.
3. Compute checksum: checksum = ~sum & 0xFF.
4. Construct frame buffer containing [data bytes ... | checksum].
5. Connect to receiver over TCP socket (127.0.0.1:6001) and transmit frame.

Checksum Receiver Algorithm

1. Create, bind, and listen on TCP port 6001.
2. Accept sender connection and receive payload frame bytes into buffer.
3. Compute 1's complement sum over all received bytes (data + checksum byte).
4. If calculated verification sum equals 0xFF, report **"RESULT: No error."**
5. Otherwise, report **"RESULT: Error detected!"**.

Checksum Sender Code (checksum_sender.c)

```
// checksum_sender.c — simple version
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 6001

unsigned char checksum(unsigned char *data, int n) {
   unsigned int sum = 0;
   for (int i = 0; i < n; i++) {
      sum += data[i];
      if (sum > 0xFF) sum = (sum & 0xFF) + 1;   // end-around carry
   }
   return ~sum & 0xFF;                          // 1's complement
}

int main() {
   char msg[] = "CheckSumTestMessage";
   int n = strlen(msg);
   unsigned char cs = checksum((unsigned char *)msg, n);

   printf("Data: %s\nChecksum: 0x%02X\n", msg, cs);

   unsigned char frame[100];
```

```
    memcpy(frame, msg, n);
    frame[n] = cs;

    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {AF_INET, htons(PORT), {inet_addr("127.0.0.1")}};
    connect(sock, (struct sockaddr *)&addr, sizeof(addr));

    send(sock, frame, n + 1, 0);
    printf("Frame sent.\n");
    close(sock);
    return 0;
}
```

Checksum Receiver Code (checksum_receiver.c)

```
// checksum_receiver.c — simple version
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 6001
#ifndef INJECT_ERROR
#define INJECT_ERROR 0
#endif

unsigned char checksum(unsigned char *data, int n) {
    unsigned int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += data[i];
        if (sum > 0xFF) sum = (sum & 0xFF) + 1;
    }
    return sum & 0xFF;
}

int main() {
    int server = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr = {AF_INET, htons(PORT), {INADDR_ANY}};
    bind(server, (struct sockaddr *)&addr, sizeof(addr));
```

```
    listen(server, 1);
    printf("Waiting for sender...\n");

    int conn = accept(server, NULL, NULL);
    unsigned char frame[100];
    int len = recv(conn, frame, sizeof(frame), 0);

    printf("Received: ");
    for (int i = 0; i < len; i++) printf("%02X ", frame[i]);
    printf("\n");

#if INJECT_ERROR
    frame[0] ^= 0x01;   // flip a bit to simulate transmission error
    printf("Injected error -> first byte now 0x%02X\n", frame[0]);
#endif

    unsigned char result = checksum(frame, len);   // includes checksum byte
    printf("Verification sum: 0x%02X\n", result);

    if (result == 0xFF)
        printf("RESULT: No error.\n");
    else
        printf("RESULT: Error detected!\n");

    close(conn);
    close(server);
    return 0;
}
```

Part B: Cyclic Redundancy Check (CRC)

Mathematical Principle

1. The generator polynomial is given as $G(x) = x^4 + x + 1$, corresponding to binary string 10011 (length $g = 5$).
2. The input data bitstream D is converted to binary (e.g., "HI" $\rightarrow$ 0100100001001001).
3. $g - 1 = 4$ zero bits are appended to the binary bitstream to obtain padded payload $D \cdot 2^{g-1}$.
4. Modulo-2 binary division (XOR operations) is performed on padded data using generator

polynomial $G(x)$:

$$\text{Remainder } R = (D \cdot 2^{g-1}) \pmod{G(x)}$$

5. Transmitted frame string $F = D \parallel R$.
6. At receiver, F is divided modulo-2 by $G(x)$. If the resulting remainder is all zeros (0000), the frame is error-free.

Algorithms

CRC Sender Algorithm

1. Convert input ASCII string to binary representation string.
2. Append $(g-1) = 4$ zero bits to binary string.
3. Execute crc_divide() via XOR modulo-2 division against generator polynomial 10011 to obtain 4-bit CRC remainder.
4. Append 4-bit CRC to original data bits to build frame string.
5. Transmit binary frame string via TCP socket (127.0.0.1:6002).

CRC Receiver Algorithm

1. Bind and listen on TCP port 6002.
2. Accept sender socket connection and receive binary frame string.
3. Compute modulo-2 division of full frame against generator 10011.
4. If remainder string consists entirely of zeros ("0000"), print **"RESULT: No error."**
5. Otherwise, report **"RESULT: Error detected!"**.

CRC Sender Code (crc_sender.c)

```
// crc_sender.c — simple version
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 6002
#define GEN  "10011"        // G(x) = x^4 + x + 1

// XOR-based binary division; returns remainder (CRC bits) in rem
void crc_divide(char *data, char *gen, char *rem) {
   int dlen = strlen(data), glen = strlen(gen);
   char temp[100];
```

```
    strcpy(temp, data);

    for (int i = 0; i <= dlen - glen; i++) {
        if (temp[i] == '1')
            for (int j = 0; j < glen; j++)
                temp[i + j] = (temp[i + j] == gen[j]) ? '0' : '1';
    }
    strcpy(rem, temp + dlen - (glen - 1));
}

int main() {
    char msg[] = "Hl";
    char bin[100] = "";

    // ASCII -> binary
    for (int i = 0; msg[i]; i++)
        for (int b = 7; b >= 0; b--)
            strcat(bin, ((msg[i] >> b) & 1) ? "1" : "0");

    int glen = strlen(GEN);
    char padded[100];
    sprintf(padded, "%s%0*d", bin, glen - 1, 0);   // append zeros

    char crc[10];
    crc_divide(padded, GEN, crc);

    char frame[100];
    sprintf(frame, "%s%s", bin, crc);

    printf("Data bits : %s\n", bin);
    printf("CRC bits  : %s\n", crc);
    printf("Frame     : %s\n", frame);

    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {AF_INET, htons(PORT), {inet_addr("127.0.0.1")}};
    connect(sock, (struct sockaddr *)&addr, sizeof(addr));
    send(sock, frame, strlen(frame), 0);
    printf("Frame sent.\n");
    close(sock);
    return 0;
}
```

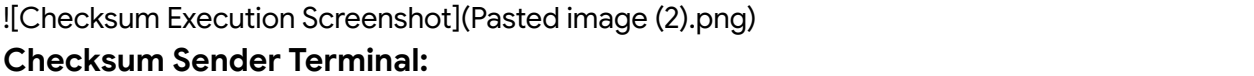
CRC Receiver Code (crc_receiver.c)

```
// crc_receiver.c — simple version
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 6002
#define GEN  "10011"
#ifndef INJECT_ERROR
#define INJECT_ERROR 0
#endif

void crc_divide(char *data, char *gen, char *rem) {
   int dlen = strlen(data), glen = strlen(gen);
   char temp[100];
   strcpy(temp, data);

   for (int i = 0; i <= dlen - glen; i++) {
      if (temp[i] == '1')
         for (int j = 0; j < glen; j++)
            temp[i + j] = (temp[i + j] == gen[j]) ? '0' : '1';
   }
   strcpy(rem, temp + dlen - (glen - 1));
}

int main() {
   int server = socket(AF_INET, SOCK_STREAM, 0);
   int opt = 1;
   setsockopt(server, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

   struct sockaddr_in addr = {AF_INET, htons(PORT), {INADDR_ANY}};
   bind(server, (struct sockaddr *)&addr, sizeof(addr));
   listen(server, 1);
   printf("Waiting for sender...\n");

   int conn = accept(server, NULL, NULL);
   char frame[100] = {0};
   int len = recv(conn, frame, sizeof(frame) - 1, 0);
   frame[len] = '\0';
   printf("Received: %s\n", frame);
```

```
#if INJECT_ERROR
   frame[3] = (frame[3] == '0') ? '1' : '0';   // flip bit 3
   printf("Injected error -> %s\n", frame);
#endif

   char rem[10];
   crc_divide(frame, GEN, rem);
   printf("Remainder: %s\n", rem);

   if (strspn(rem, "0") == strlen(rem))
      printf("RESULT: No error.\n");
   else
      printf("RESULT: Error detected!\n");

   close(conn);
   close(server);
   return 0;
}
```

Code Output & Execution Screenshots

Checksum Test Output



Checksum Sender Terminal:

```
[sarav@vivobook-s14-m5406w Exp6]$ ./chcksumsend
Data: CheckSumTestMessage
Checksum: 0x80
Frame sent.
```

Checksum Receiver Terminal:

```
[sarav@vivobook-s14-m5406w Exp6]$ ./chcksumrecv
Waiting for sender...
Received: 43 68 65 63 6B 53 75 6D 54 65 73 74 4D 65 73 73 61 67 65 80
Verification sum: 0xFF
RESULT: No error.
```

CRC Test Output

![CRC Execution Screenshot](Pasted image_2.png)

CRC Sender Terminal:

```
[sarav@vivobook-s14-m5406w Exp6]$ ./crcsend
Data bits : 0100100001001001
CRC bits  : 0110
Frame     : 01001000010010010110
Frame sent.
```

CRC Receiver Terminal:

```
[sarav@vivobook-s14-m5406w Exp6]$ ./crcrecv
Waiting for sender...
Received: 01001000010010010110
Remainder: 0000
RESULT: No error.
```

Comparative Analysis: Checksum vs. CRC

Criterion	1's Complement Checksum	Cyclic Redundancy Check (CRC)
Mathematical Basis	Binary 1's complement addition with carry	Polynomial modulo-2 division
Error Detection Power	Detects single-bit errors; fails for equal opposing bit flips	Detects all single, double, odd-numbered bit errors and burst errors up to generator length
Hardware / Software Overhead	Extremely lightweight; fast addition in software	Requires XOR shifts; highly hardware-efficient
Layer Usage	Transport / Network Layer (IP, TCP, UDP checksums)	Data Link Layer (Ethernet frames)

Learning Outcomes

1. Implemented 1's complement addition with end-around carry for sender checksum calculation and receiver validation.
2. Constructed binary polynomial modulo-2 division to compute and check 4-bit CRC

remnants using generator $G(x) = x^4 + x + 1$.

3. Transmitted error-detection frames over TCP sockets and verified error-free reception versus bit-flip error scenarios.